

Performance Intelligence Copilot



IA aplicada para automatizar el análisis y acelerar decisiones sobre pruebas de rendimiento

Grupo 8

Luis Araya
Rodrigo González
Hernán Medina

Profesor guía: Ahmad Armoush · Agosto 2026

01 · CONTEXTO Y EVOLUCIÓN DEL PROYECTO

Del negocio bancario a una decisión de rendimiento explicable

NEGOCIO BANCARIO

Personas y empresas

Acceden a productos y servicios financieros a través de canales que deben responder de forma continua y estable.

WEB

Canales digitales

APP

Operaciones

API

Servicios integrados

CAPACIDAD HABILITANTE

Pruebas de rendimiento

Permiten medir tiempos de respuesta, errores y capacidad antes de liberar cambios.

CAMBIO DE ENFOQUE

PRIMERA PRESENTACIÓN

Predicción de capacidad e infraestructura cloud

El planteamiento era más amplio que los datos y capacidades demostrables del proyecto.



PROYECTO REAL

Copilot para analizar pruebas Gatling

Integra configuración, resultados e historial para entregar una recomendación explicable.

EVIDENCIA REAL

Configuración → métricas → análisis → recomendación → validación humana

Así llegamos al problema: la prueba puede ejecutarse, pero preparar la evidencia, interpretarla y acordar la siguiente acción sigue requiriendo tiempo y criterio especializado.

3.960 casos/año en 11 pipelines: el cuello de botella está en la interpretación

3.960

casos de prueba / año

La operación procesa 330 casos mensuales en 11 pipelines. El costo recurrente de diseño y ejecución alcanza 4,8 UF por prueba, acumulando 19.008 UF anuales en esfuerzo operativo.

330/mes

casos Gatling en 11 pipelines

86 %

del costo en automatización y mantención

VOLUMEN OPERATIVO	COSTO ANUAL ESTIMADO	COSTO PROMEDIO / CASO	DESGLOSE DE ESFUERZO
3.960 casos / año · 330 × 12 meses · 11 pipelines activos	19.008 UF operación directa (diseño + ejecución) · ≈ \$718 MM/año	4,8 UF diseño 2,4 UF + ejecución 2,4 UF · ≈ \$181 mil por prueba	34,5 UF ciclo completo / caso automatización 19,8 UF + mantención 9,9 UF = 86 % del total

Acelerar la decisión técnica: de horas a minutos

OBJETIVO GENERAL

Integrar configuración, resultados e historial Gatling para recomendar la siguiente acción técnica con trazabilidad y validación humana.

01

Pipeline confiable

Ingesta, validación y normalización reproducible.

02

Motor explicable

Recomendación maintain/review/evolve con reglas y modelo interpretable.

03

Evaluación controlada

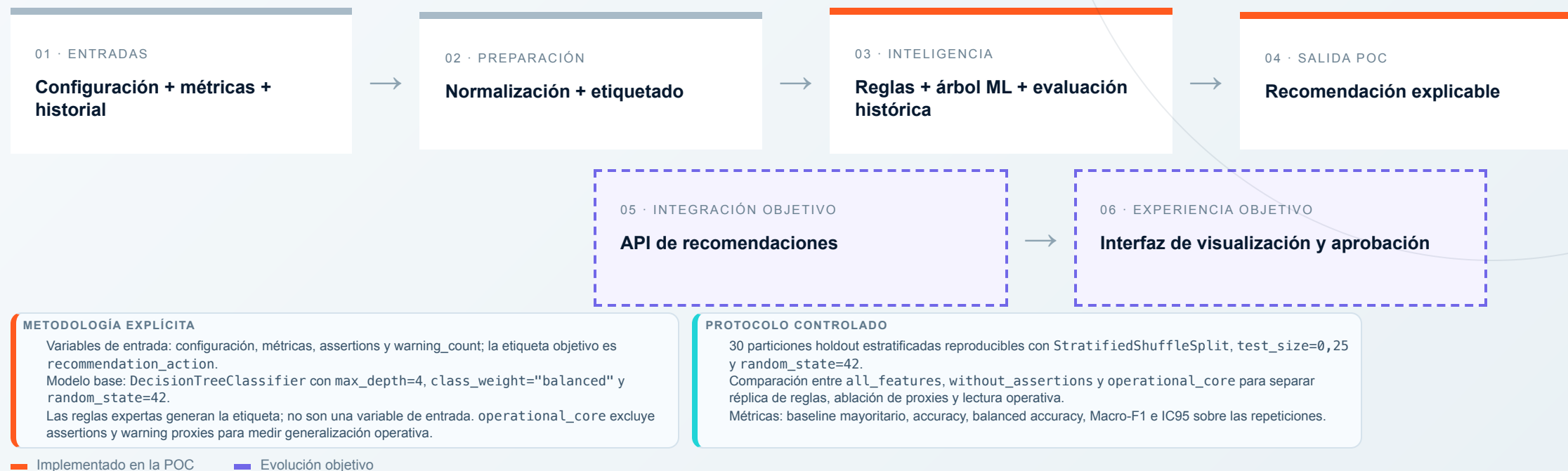
Comparación con baseline y control explícito de sobreajuste.

KPI	META	RESULTADO (OPERATIONAL_CORE)	ESTADO	Cobertura	>= 95 % 96,6 % (28/29) Cumple	Macro-F1 operacional	>= 0,55 0,5987 Cumple	Balanced accuracy operacional	>= 0,55 0,62 Cumple
Desv. Macro-F1 operacional	<= 0,20	0,1738	Cumple	Mejora Macro-F1 vs baseline	>= 0,15 +0,1820 pp Cumple				

Las variantes cercanas a las reglas (all_features, without_assertions) alcanzaron 1,00, pero operational_core se usa como resultado principal por excluir proxies cercanos a la etiqueta.

04 · METODOLOGÍA Y ARQUITECTURA

Un flujo híbrido desde la evidencia hasta la recomendación



Composición y calidad del dataset validado

La muestra útil es pequeña: 28 registros. El CSV no trae microservicio explícito, por lo que la procedencia se reconstruye desde el árbol de orígenes de ejemplo.

EMBUDO

29 → 28

1 ejecución fue omitida por duplicado.

CLASE

20 / 8

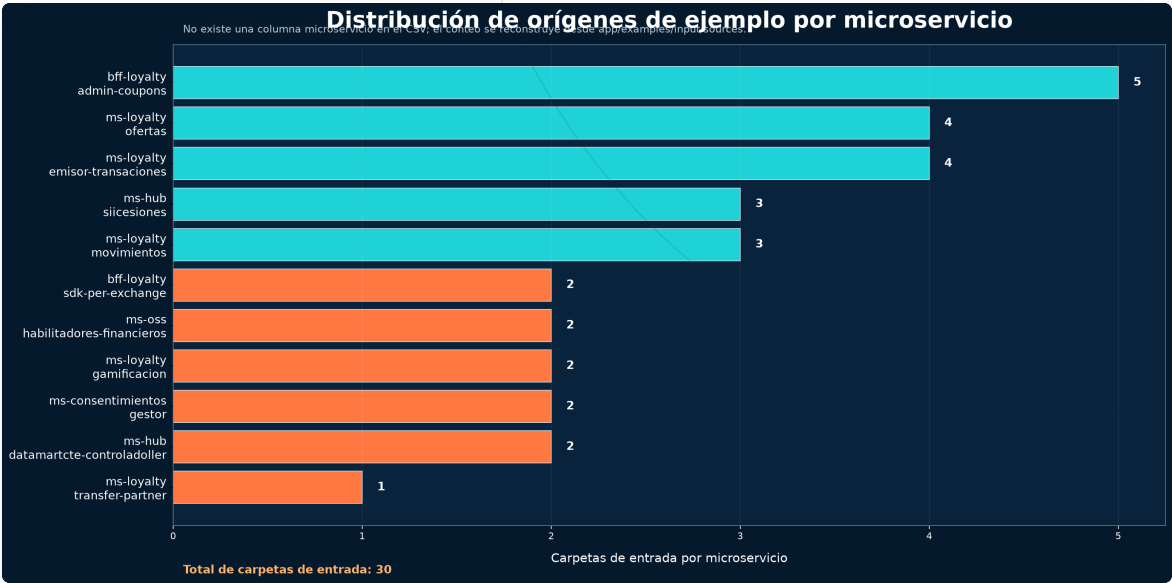
La muestra está desbalanceada.

NULOS

1 columna

p90_response_time_ms quedó vacía.

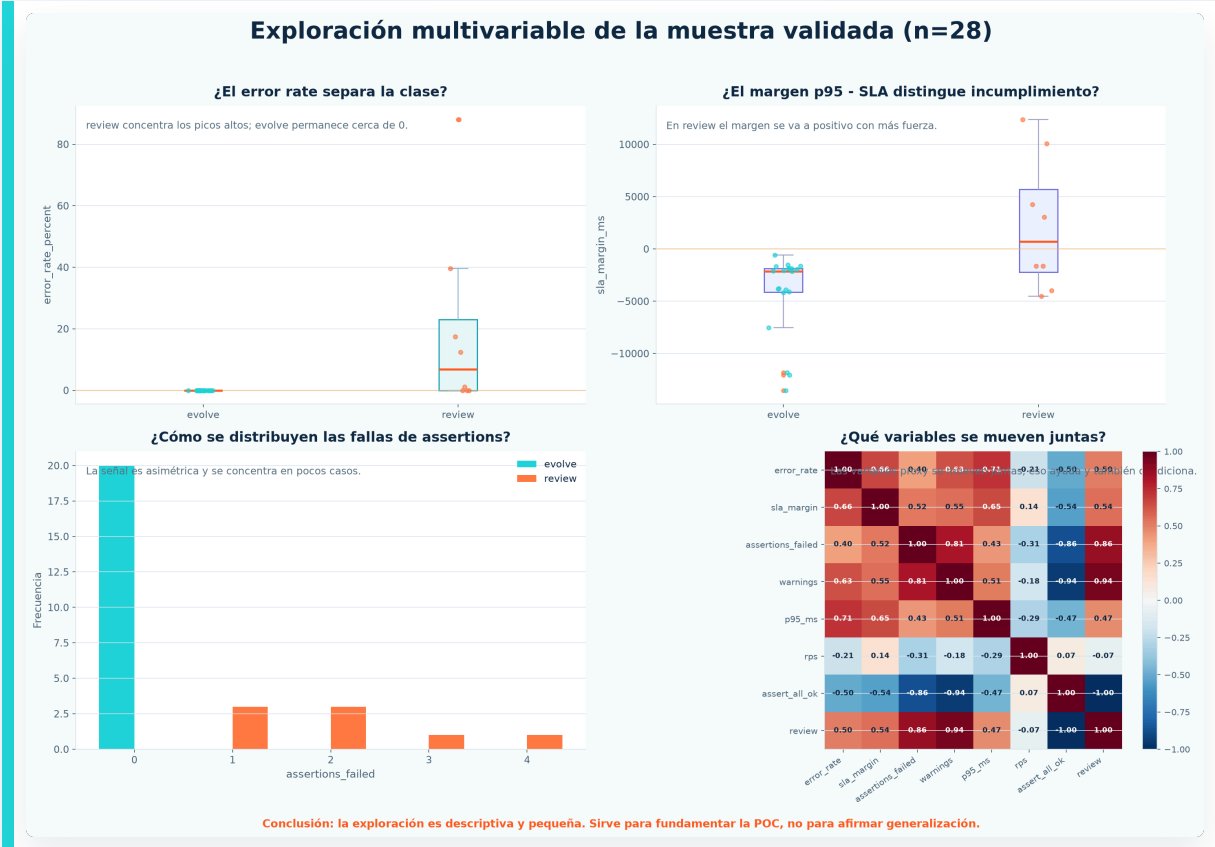
VARIABLE	PREGUNTA QUE RESPONDE	ROL
error_rate_percent	Señal de falla operacional	Señal
p95_response_time_ms	Latencia del 95 % de solicitudes	Señal
sla_margin_ms	Brecha entre p95 y SLA	Derivada
assertions_failed	Reglas técnicas incumplidas	Señal
warning_count	Advertencias del análisis	Proxy
recommendation_action	Salida: maintain, review o evolve	Etiqueta



Fuente: árbol app/examples/input/sources. Se usan 30 carpetas de entrada repartidas en 11 microservicios; el CSV final sigue siendo de 28 registros.

Las variables separan la clase y muestran proxies

Cada visual responde una pregunta distinta. Con n=28, la exploración es descriptiva y sirve para fundamentar la POC, no para probar un universo distinto.



03 · ASSERTIONS FAILED

las fallas se concentran en pocos casos

La señal es asimétrica y confirma que las reglas pesan, aunque aparecen poco.

04 · CORRELACIONES

hay coherencia interna, no causalidad

Las variables proxy se mueven juntas; eso ayuda al modelo y también lo condiciona.

ETL e ingeniería de atributos para construir variables auditables

EMBUDO

29 → 28

Se omitió 1 ejecución duplicada; la muestra útil queda en 28 registros.

DISTRIBUCIÓN

20 / 8

El corte final está desbalanceado, por eso se evalúa de forma estratificada.

NULOS

1 columna

p90_response_time_ms quedó vacía en todas las filas y se excluyó del entrenamiento.

CONTROL	QUÉ SE HIZO	POR QUÉ IMPORTA
Outliers	No se aplicó un detector automático. Los extremos se revisaron en la exploración y se conservaron porque son ejecuciones reales y el dataset es pequeño.	Evita perder señal operativa y mantiene representatividad.
Fuga de información	Se excluyen <code>schema_version</code> , <code>metrics_scope</code> y la etiqueta <code>recommendation_action</code> . En <code>operational_core</code> también se quitan <code>assertions_*</code> y <code>warning_count</code> .	Impide que el modelo vea metadatos o proxies demasiado cercanos a la decisión.
Historial comparable	Es la misma estructura de columnas, el mismo esquema y el mismo pipeline de preparación para comparar ejecuciones sin cambiar la regla de lectura.	Permite comparar corridas con una base homogénea y trazable.

PREPROCESAMIENTO REAL ANTES DE ENTRENAR

01

Filtrar columnas válidas
Se usan solo las variables no metadata y no etiqueta. Luego se elimina la columna vacía `p90_response_time_ms`.

02

Imputar faltantes
Las numéricas usan mediana; las categóricas usan el valor `__missing__`.

03

Codificar categóricas
`load_type` y `assertions_all_passed` pasan por `OneHotEncoder`. No se usa escalamiento.

04

Entrenar el árbol
El modelo final es `DecisionTreeClassifier` con `max_depth=4`, `class_weight="balanced"` y `random_state=42`.

Margen SLA: `sla_margin_ms = p95_response_time_ms - strictest_response_time_target_ms`. Si el resultado es positivo, el p95 supera el SLA; si es negativo, cumple.

Qué aporta hoy el árbol y qué deberá aprender después

Evidencia	Qué permite afirmar	Qué no permite afirmar
Variantes cercanas a reglas	all_features y without_assertions llegan a Macro-F1 = 1,00 y balanced accuracy = 1,00 en 30 particiones	Eso sí reproduce etiquetas
Señales operacionales	operational_core baja a Macro-F1 = 0,5987 y balanced accuracy = 0,62	La señal operativa sola es más débil
Lectura metodológica	Las variables cercanas al motor de reglas reproducen las etiquetas; al restringirlas a señales operacionales, el desempeño cae	No debe leerse como reproducción fuerte de reglas expertas

APORTE DEL MACHINE LEARNING · CONVERGENCIA CON REGLAS EXPERTAS, NO SUSTITUCIÓN

IMPORTANCIA APRENDIDA POR EL ÁRBOL · operational_core (sin proxies)

VARIABLE	IMPORTANCIA
ROL OPERACIONAL	
error_rate_percent	Porcentaje de solicitudes fallidas en la ejecución
min_response_time_ms	Tiempo mínimo de respuesta; señala degradación base

Fuente: historical_model_explanation.json · importancia Gini del árbol entrenado sobre el perfil operational_core (18 variables, sin assertions ni warning_count).

HALLAZGO DE ESTA MUESTRA

Sin proxies de reglas, el árbol se apoya en error_rate_percent y min_response_time_ms con igual importancia.

Esto sugiere que la tasa de error y la latencia mínima son las señales operativas más informativas disponibles para separar las clases en esta muestra.

SIGUIENTE VALIDACIÓN

Desacoplar etiquetas y señales operacionales

Separar entrenamiento/prueba por microservicio y usar etiquetas expertas independientes para medir generalización real.

Las variantes cercanas a reglas superan a las operacionales

CONCLUSIÓN DEL EXPERIMENTO

La comparación separa señales cercanas a reglas de señales operacionales

Las variantes con variables cercanas al motor de reglas alcanzan 1,00; al restringirlas a señales operacionales, el desempeño desciende a Macro-F1 0,5987 y balanced accuracy 0,62.

DATOS EVALUADOS

28 ejecuciones

20 evolve · 8 review

OPERATIONAL_CORE

Macro-F1 0,5987

balanced accuracy 0,62 · review F1 0,3968 · std 0,1738

PROTOCOLO

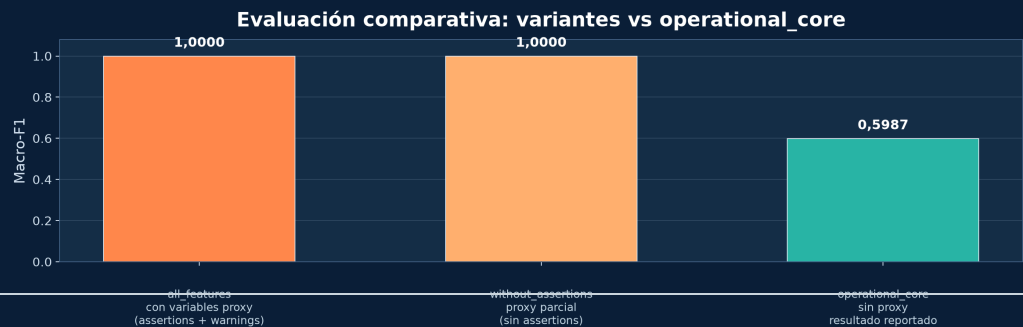
30 particiones

holdout estratificado · 21/7 · random_state 42

BASELINE MAYORITARIO

Macro-F1 0,4167

accuracy 0,7143 · balanced acc. 0,50 · review F1 0,00



Lectura: los dos 1,0000 corresponden a variantes con señales proxy; operational_core (sin proxy) reporta 0,5987.

10 · INTEGRACIÓN EN EL PIPELINE

La recomendación espera aprobación antes de ejecutar Gatling



11 · IMPACTO OPERACIONAL ESPERADO

Impacto operacional y evaluación económica preliminar

ESCENARIO A · ACTUAL

4,8 UF / prueba

Diseño 2,4 UF + Ejecución 2,4 UF por caso (≈ \$181 mil CLP). Costo recurrente que el Copilot busca reducir.



ESCENARIO B · CON COPILOT

Minutos + validación humana

La preparación analítica y la recomendación se automatizan; QA recibe evidencia explicable para aprobar con mayor rapidez.

BASE HISTÓRICA / ANUAL

19.008 UF

$3.960 \text{ casos/año} \times 4,8 \text{ UF} \approx \$718,5 \text{ MM CLP/año}$

ALCANCE DE COBERTURA

3.960 casos/año

$11 \text{ pipelines} \times 330 \text{ casos/mes} \cdot \text{diseño } 2,4 \text{ UF} + \text{ejecución } 2,4 \text{ UF}$

ESCENARIO PRELIMINAR

7.128 UF / año

$19.008 \text{ UF} \times 50 \% \text{ cobertura} \times 75 \% \text{ reducción} \approx \$269,4 \text{ MM CLP/año}$

Una base explicable para decisiones de rendimiento más consistentes

CONTRIBUCIÓN

La POC transforma resultados Gatling en datos estructurados y demuestra que un árbol interpretable puede aprender las etiquetas del motor experto cuando utiliza variables cercanas a esas reglas. Al restringirlo a señales operacionales, **supera el baseline**, aunque su generalización aún debe validarse con nuevos datos y etiquetas independientes.

Siguiente validación

- Validar las recomendaciones con especialistas
- Ampliar el histórico con nuevos casos etiquetados
- Medir tiempo de análisis y reejecuciones en un piloto

La generalización deberá validarse con nuevos datos y revisión experta.